

Morent -сервис аренды авто

**Никитин Р. А.
Шарифуллин А. В.
Юсупов Д. А.
Азмитов А. Ш.
Гарифуллин А. А.**

Morent: сервис аренды автомобилей

Коротко: удобный, прозрачный, быстрый сервис аренды авто — от туриста до корпоративного клиента. Видение фокусируется на клиентском опыте, прозрачности цен и масштабируемой платформе, готовой к новым сервисам мобильности.



Функциональные требования

ID	Сервис	Приоритет	Требование
FR-01	Morent core	Must	Платформа должна поддерживать регистрацию, вход и сессию пользователя.
FR-02	Morent core	Must	Платформа должна разделять доступ по ролям user и admin.
FR-03	Morent core	Must	Платформа должна отдавать каталог авто с фильтрами и страницей деталей.
FR-04	Morent core	Must	Платформа должна создавать аренду и запрещать пересечение броней одного авто.
FR-05	Morent core	Must	Платформа должна поддерживать избранное, комментарии и проверку "комментарий только после аренды".
FR-06	Morent core	Must	Админ-панель должна поддерживать CRUD по доменным сущностям и аудит действий.
FR-07	Morent core	Must	Сервис должен поддерживать загрузку медиа в объектное хранилище и выдачу URL.
FR-08	car-aggregator	Must	Агрегатор должен выполнять поиск авто по запросу с валидацией через DaData и данными CarAPI.
FR-09	car-aggregator	Must	Агрегатор должен исключать из результатов машины, уже "забранные" в offers.
FR-10	car-aggregator	Must	Агрегатор должен поддерживать регистрацию взятия/освобождения оффера (purchase/release).
FR-11	car-aggregator	Should	Агрегатор должен поддерживать fallback-стратегии поиска (exact/fuzzy/dadata).
FR-12	user-system	Must	Сервис пользователей должен поддерживать JWT-аутентификацию и управление пользователями (CRUD).
FR-13	user-system	Must	Сервис пользователей должен поддерживать RBAC: роли, права, назначения прав ролям.
FR-14	user-system	Must	Сервис пользователей должен поддерживать управление компаниями и привязку пользователей.
FR-15	user-system	Should	Сервис пользователей должен публиковать асинхронные события через RabbitMQ.

Функциональные требования

FR-16	payment-service	Must	Платежный сервис должен поддерживать создание счетов, пополнение, списание и переводы.
FR-17	payment-service	Must	Платежный сервис должен обеспечивать идемпотентность переводов по ключу операции.
FR-18	payment-service	Must	Платежный сервис должен поддерживать лимиты, историю операций и реверс перевода.
FR-19	payment-service	Should	Платежный сервис должен поддерживать batch-переводы и статистику по переводам.
FR-20	EmailService	Must	Email API должен принимать запрос на отправку, валидировать данные и ставить письмо в очередь.
FR-21	EmailService	Must	Worker должен асинхронно обрабатывать очередь, рендерить шаблон и отправлять письмо через провайдера.
FR-22	EmailService	Must	Сервис должен хранить и отдавать статус доставки письма по correlationId.
FR-23	EmailService	Should	Сервис должен принимать webhook статусы от провайдера с HMAC-проверкой подписи.
FR-24	generator-service	Must	Сервис генерации должен выдавать пароль и пароль по маске через HTTP API.
FR-25	generator-service	Must	Сервис генерации должен выдавать QR-код PNG по входной строке и размеру.
FR-26	generator-service	Should	Сервис генерации должен валидировать параметры маски и размера с понятными ошибками.
FR-27	Интеграция	Must	Morent должен интегрироваться с агрегатором для поиска/импорта авто в админке.
FR-28	Интеграция	Must	Morent должен использовать payment-service для создания и проверки статуса платежа аренды.
FR-29	Интеграция	Must	Morent должен инициировать email-уведомления через EmailService по ключевым событиям аренды.
FR-30	Интеграция	Should	Morent должен иметь возможность использовать user-system как внешний IAM-источник.

Нефункциональные требования

ID	Сервис	Категория	Требование
NFR-01	Morent core	Availability	Доступность API Morent на показе хотя бы быть
NFR-02	Morent core	Performance	Для ключевых REST-операций Morent p95 latency должна быть <= 400 ms при нагрузке до 50 RPS.
NFR-03	Morent core	Reliability	Система не должна допускать двойную аренду на пересекающиеся интервалы.
NFR-04	Morent core	Security	Доступ к admin endpoints должен быть ограничен ролью admin.
NFR-05	car-aggregator	Performance	Поисковый запрос агрегатора должен обрабатываться с p95 <= 1500 ms при нагрузке до 15 RPS.
NFR-06	car-aggregator	Reliability	При недоступности DaData/CarAPI должен срабатывать fallback без падения сервиса.
NFR-07	user-system	Security	JWT-токены должны проверяться на каждом защищенном endpoint.
NFR-08	user-system	Reliability	Ошибки RBAC должны возвращаться в единообразном формате (401/403).
NFR-09	payment-service	Consistency	Операции перевода должны быть идемпотентны и консистентны при повторных запросах.
NFR-10	payment-service	Performance	p95 для денежных операций должна быть <= 300 ms при нагрузке до 80 RPS в памяти.
NFR-11	EmailService	Reliability	Очередь отправки должна поддерживать retry при временных ошибках SMTP/API провайдера.
NFR-12	EmailService	Security	Вебхуки статусов должны проходить проверку HMAC-подписи.
NFR-13	generator-service	Performance	Генерация пароля должна выполняться <= 100 ms, QR <= 300 ms для размера 256x256.
NFR-14	generator-service	Security	Сервис не должен логировать чувствительные значения сгенерированных паролей в открытом виде.
NFR-15	Все сервисы	Deployability	Порты и параметры запуска должны подтягиваться из .env и docker compose без хардкода.
NFR-16	Все сервисы	Observability	Для каждого сервиса должны быть health/readiness checks и технические логи.
NFR-17	Все сервисы	Maintainability	Сервисы должны сохранять слоистую структуру (transport/service/repository/domain).
NFR-18	Все сервисы	Recoverability	Целевой RPO для БД/состояния — до 24 часов, RTO — до 30 минут

Test-cases

ТС-03 Успешное создание аренды

- Предусловия:

1. Пользователь авторизован.
2. Автомобиль существует и свободен в выбранный период.

- Шаги:

1. Отправить запрос на создание аренды с валидными датами.
2. Получить ответ API.

- Ожидаемый результат:

1. Возвращается `201 Created`.
2. В ответе содержится созданная аренда с корректными датами и `car\_id`.

ТС-04 Запрет двойной аренды

- Предусловия:

1. Для авто уже существует аренда на период `A-B`.

- Шаги:

1. Отправить новый запрос на аренду того же авто на пересекающийся период.

- Ожидаемый результат:

1. Возвращается ошибка бизнес-валидации.
2. Вторая аренда не создается.

Test-cases

ТС-08 Поиск в агрегаторе и импорт авто

- Предусловия:

1. Админ авторизован.
2. Сервис агрегатора доступен.

- Шаги:

1. Выполнить запрос поиска в агрегатор.
2. Выбрать один из результатов.
3. Выполнить импорт в Morent.

- Ожидаемый результат:

1. API возвращает успешный ответ.
2. Импортированное авто появляется в каталоге.

ТС-09 Недоступный агрегатор

- Предусловия:

1. Админ авторизован.
2. Агрегатор недоступен.

- Шаги:

1. Выполнить запрос поиска в агрегатор.

- Ожидаемый результат:

1. Возвращается контролируемая ошибка (`5xx/Service Unavailable`).
2. Основной backend Morent остается работоспособным.

Test-cases

ТС-12 Проверка ролей для админ-операций

- Предусловия:

1. Есть токен обычного пользователя.

- Шаги:

1. Отправить запрос на admin endpoint с user-токеном.

- Ожидаемый результат:

1. Возвращается `403 Forbidden` (или `404 Not Found` чтобы не 'палить' админ-панель)

2. Доступ к админ панели закрыт

ТС-20 Импорт авто из агрегатора в Morent

- Предусловия:

1. Админ Morent авторизован.

2. `car-aggregator` доступен.

- Шаги:

1. Выполнить поиск авто в агрегаторе из админки Morent.

2. Выбрать найденный авто-оффер.

3. Выполнить импорт в каталог Morent.

- Ожидаемый результат:

1. Импорт завершается успешно.

2. Автомобиль появляется в каталоге Morent.

3. Повторный поиск в агрегаторе не показывает уже забранный оффер.

Use-cases

UC-01 Регистрация и вход пользователя

- Цель: получить доступ к личному кабинету и операциям аренды.
- Предусловия: пользователь не авторизован.
- Основной сценарий:
 1. Пользователь отправляет форму регистрации.
 2. Система создает пользователя.
 3. Пользователь выполняет вход.
 4. Система выдает токен/сессию.
- Негативные альтернативы:
 1. Email уже существует — регистрация отклоняется.
 2. Неверный пароль — вход отклоняется.

UC-02 Поиск и бронирование автомобиля

- Цель: создать аренду на выбранный период.
- Предусловия: пользователь авторизован.
- Основной сценарий:
 1. Пользователь фильтрует каталог и выбирает авто.
 2. Вводит даты аренды.
 3. Система проверяет пересечения бронирований.
 4. Система создает аренду.
- Негативные альтернативы:
 1. Даты некорректны (`end <= start`) — запрос отклоняется.
 2. Есть пересечение с существующей бронью — запрос отклоняется.

Use-cases

UC-03 Добавление автомобиля в избранное

- Цель: сохранить интересующие авто для быстрого доступа.
- Предусловия: пользователь авторизован.
- Основной сценарий:
 1. Пользователь нажимает "добавить в избранное" в карточке авто.
 2. Система создает связь `user-car`.
 3. Пользователь видит авто в списке избранного.
- Негативные альтернативы:
 1. Авто не существует — операция отклоняется.
 2. Повторное добавление не создает дубль.

UC-04 Импорт автомобиля из агрегатора

- Цель: пополнить каталог Morent через внешний агрегатор.
- Предусловия: администратор авторизован.
- Основной сценарий:
 1. Администратор отправляет поисковый запрос в агрегатор.
 2. Получает список подходящих авто.
 3. Выбирает авто и инициирует импорт.
 4. Авто появляется в каталоге Morent.
- Негативные альтернативы:
 1. Агрегатор недоступен — отображается контролируемая ошибка.
 2. Некорректные данные ответа агрегатора — импорт отклоняется.

Use-cases

UC-05 Оплата аренды

- Цель: подтвердить аренду через платежный сервис.
- Предусловия: пользователь на этапе создании аренды
- Основной сценарий:
 1. Morent создает запрос в payment service.
 2. Payment service обрабатывает платеж.
 3. Morent получает итоговый статус оплаты.
 4. Пользователь видит подтверждение оплаты.
- Негативные альтернативы:
 1. Платеж отклонен — аренда отклоняется.
 2. Payment service недоступен — операция переводится в retry/pending.

UC-06 Оставить комментарий после аренды

- Цель: оставить отзыв только при реальном опыте аренды.
- Предусловия: пользователь авторизован.
- Основной сценарий:
 1. Пользователь отправляет комментарий и рейтинг.
 2. Система проверяет факт аренды этого авто.
 3. Комментарий сохраняется и отображается в карточке авто.
- Негативные альтернативы:
 1. Аренды не было — возвращается `403 Forbidden`.

Use-cases

UC-07 Просмотр и модерация данных в админке

- Цель: операционное управление платформой.
- Предусловия: пользователь с ролью `admin`.
- Основной сценарий:
 1. Администратор открывает админ-панель.
 2. Выполняет CRUD-операции по авто/пользователям/комментариям.
 3. Система сохраняет изменения и записывает аудит-лог.
- Негативные альтернативы:
 1. Не-админ пытается выполнить операцию — доступ запрещен (403/404).

UC-08 Управление ролями и правами

- Цель: централизованно управлять доступом пользователей.
- Предусловия: администратор user-system авторизован.
- Основной сценарий:
 1. Администратор создает/обновляет роль.
 2. Назначает роли набор прав.
 3. Привязывает роль к пользователю.
 4. Изменения вступают в силу для защищенных endpoint.
- Негативные альтернативы:
 1. Попытка удалить системную роль отклоняется.
 2. Некорректный набор permission возвращает ошибку валидации.